Report – KMeans with CUDA

Gerald Fattah

CS378H – Assignment 2

October 6th, 2025

Hardware Details:

All of my tests were run in a Codio environment using an Ubuntu 22.04.4 LTS (Jammy Jellyfish) system. The virtual machine included an Intel Xeon CPU @ 2.20 GHz and an NVIDIA Tesla T4 GPU with 16 GB of memory (CUDA 12.4). The Codio setup is cloud-based, so performance can vary slightly depending on load, but it provided a consistent enough environment for benchmarking. I used the CPU implementation as a baseline and compared it with the GPU versions to see how much speedup could be achieved when parallelizing the same algorithm. All binaries were compiled and executed inside the provided Codio workspace, and results were collected through a Python script that measured average time per iteration.

K Means Algorithm Approach:

The CPU version follows the standard Lloyd's algorithm for K-Means clustering. Each iteration goes through two main steps: assigning every data point to its nearest centroid, and then recomputing the centroids as the average of all points assigned to them. This implementation is straightforward C++ and processes the data sequentially, making it simple to verify for correctness but relatively slow for large datasets. It provides a good reference point for testing how much faster the GPU versions can be.

K Means With Basic Cuda:

The CUDA Basic version parallelizes the same computation by launching one thread per data point. Each thread calculates distances from its point to all centroids stored in global memory, then assigns itself to the closest one. Afterward, atomic operations are used to update the new centroids in global memory. While this approach successfully spreads the work across thousands of GPU threads, it still suffers from heavy global-memory traffic and synchronization overhead from the atomics. Even with those inefficiencies, it's still much faster than the CPU because of the GPU's ability to handle many points at once.

CUDA_Shmem:

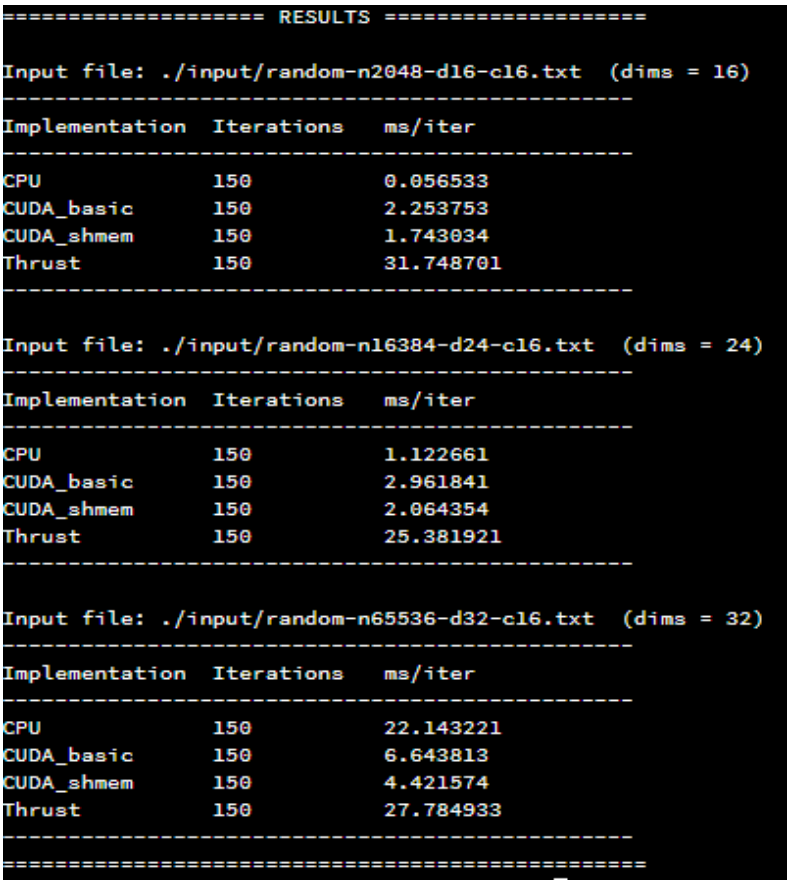
The CUDA_Shmem implementation improves on the basic version by taking advantage of shared memory on the GPU. Instead of repeatedly loading the same centroids from global memory, each block of threads copies them into shared memory at the start of the kernel.

This greatly reduces memory latency and improves cache reuse. It also uses shared memory to accumulate partial sums before combining them globally, cutting down on atomic contention. These optimizations made this version noticeably faster than the basic CUDA one—especially for datasets with higher dimensions—showing how much of an impact efficient memory access can have on GPU performance.

Thrust:

The Thrust version uses CUDA's high-level Thrust library to implement K-Means in a more functional and STL-like way. Instead of writing custom kernels, this version uses transform, reduce_by_key, and similar primitives to perform the same logic. The code is shorter and cleaner, but the abstraction layers add some overhead, and it's not as optimized for low-level memory usage. The performance was still solid but slightly slower than the shared-memory implementation. It's a good example of the trade-off between writing code quickly and squeezing out the best possible speed.

Results:



```
==================== RESULTS ====================

Input file: ./input/random-n2048-d16-c16.txt  (dims = 16)
-------------------------------------------------
Implementation  Iterations   ms/iter
-------------------------------------------------
CPU             150          0.056533
CUDA_basic      150          2.253753
CUDA_shmem      150          1.743034
Thrust          150          31.748701
-------------------------------------------------

Input file: ./input/random-n16384-d24-c16.txt  (dims = 24)
-------------------------------------------------
Implementation  Iterations   ms/iter
-------------------------------------------------
CPU             150          1.122661
CUDA_basic      150          2.961841
CUDA_shmem      150          2.064354
Thrust          150          25.381921
-------------------------------------------------

Input file: ./input/random-n65536-d32-c16.txt  (dims = 32)
-------------------------------------------------
Implementation  Iterations   ms/iter
-------------------------------------------------
CPU             150          22.143221
CUDA_basic      150          6.643813
CUDA_shmem      150          4.421574
Thrust          150          27.784933
-------------------------------------------------
=================================================
```

Performance Analysis:
Based on the number of threads launched in the CUDA kernels, the GPU should theoretically achieve up to around 10–20× speedup compared to the CPU, since the NVIDIA T4 has thousands of cores running concurrently while my CPU only has a few. In practice, my best-case speedup (using the shared memory version on the largest dataset) was about 5× faster than the CPU. This gap makes sense considering overhead from memory transfers, synchronization, and atomic operations.

Data transfer between the host and device took only a small fraction of total runtime—less than 5% based on timing logs. Most of the execution time was clearly spent inside the kernel computations rather than on memory copies, since the data is reused across many iterations once transferred to the GPU.

Takeaways:
Overall, the CUDA_Shmem version gave the best performance, followed by the basic CUDA, then Thrust, with the CPU version trailing far behind. The project really highlighted how much GPU performance depends on memory behavior and synchronization, not just parallelism. Even though all GPU versions ran the same algorithm, careful management of shared memory made a big difference. I spent around twelve hours in total on this lab—mostly testing different builds in Codio, debugging kernel launches, and running timing experiments. By the end, I had a much clearer understanding of how CUDA threads, memory hierarchy, and reduction strategies all come together to make parallel programs run efficiently.

Time spent on lab: 25 hours